

Group 5 Final Report:

Who Said That? *The Office* Character Attribution

Harsh Upadhyay, Kyen So, Vedant Goyal

{upadhh1, sok13, goyalv3}@mcmaster.ca

1 Introduction

Our task is authorship attribution, where we predict which character has the highest probability to have spoken a particular line of dialogue. We will focus on a single television series, the US adaptation of the popular sitcom, *The Office*, to ensure the feasibility of our scope. By exploring dialogues from a sitcom, we are able to explore how vocabulary choices, types of humour, tone, character dynamics, and speech patterns can be used to distinguish between characters. For the models, we will be dealing with *multi-class classification*.

2 Related Work

Authorship attribution has been widely studied within the field of Natural Language Processing, traditionally focusing on identifying the author of longer documents such as essays or books. Early approaches (Stamatatos, 2021) relied on stylometric features like word frequency and sentence structure, showing that writing style alone can often distinguish authors. This connects directly to our work, as we also rely on linguistic patterns, but at a much smaller scale using short dialogue lines instead of full documents.

More recent work has shifted toward machine learning and deep learning models that learn patterns directly from text. For example, some approaches (Fabien et al., 2022) combine multiple linguistic features, while others (Tyo and Bhatt, 2024) use neural networks like CNNs and RNNs to capture phrasing and context. These methods show that models can detect subtle differences in expression, which is central to our task, though most focus on longer texts, making our short dialogue setting more challenging.

A closely related problem to our work is character identification from dialogue in fictional

media. Venkatesh Gopal explored this using shows like *Friends* and *South Park*, but found that models performed no better than random. This highlights how difficult it is to rely only on dialogue, since humans naturally use context and character knowledge.

Overall, prior work shows that while authorship attribution works well for longer text, applying it to short dialogue remains challenging. Our work builds on this by exploring whether modern methods can still capture the subtle differences in how characters speak in *The Office*.

3 Dataset

As discussed in our project proposal, we have decided to refer to a vast collection of curated quotes from the episodes of *The Office*. This collection is curated by Max Wiseman and it's listed at: <https://quotes.maxwiseman.io/episodes> (Wiseman, 2024).

Disclaimer: Please keep in mind *The Office* is rated as TV-14 in Canada. The following dataset will contain dialogue that are suggestive and it may include mild profanity, and inappropriate or sexual jokes/references.

3.1 Data Pre-Processing and Formatting

As stated in our earlier report, we were lucky to find Max Wiseman's public repository, which contains the mdx files consisting of dialogues per episode. Using a Python script we made, we extracted every line from each file, excluding action phrases like ["he whistles"] or ["she points at him"] as well as other "mdx code" that remained as text. We also filtered through the dialogue by a certain set of characters that we have decided were significant, by using regular expressions while verifying the character we want from this list: (*Michael, Dwight,*

Jim, Pam, Ryan, Andy, Angela, Kevin, Oscar, Stanley, Phyllis, Meredith, Creed, Kelly, Toby, Darryl, Erin) We did not include the minor characters or guest characters.

3.2 Dataset Description

The dataset contains 49065 instances of dialogues sorted in 2 columns (*Name and Dialogue*). Here is a sample of **dialogue.csv**:

Name	Dialogue
Jim	What do we got here?
Dwight	Mega-Desk.
Jim	Of course.
Dwight	Command central.

4 Features

For all of our models, we use dialogue text as input, which is converted into contextual word embeddings using the pretrained models. These embeddings are not static and are learned as part of the network, allowing the models to capture meaning, tone, and phrasing. To make this stronger, we include nearby dialogue as context so each line is not treated in isolation, and we add simple structural features like sentence length and punctuation to capture how characters speak, not just what they say. We do not perform explicit feature selection, instead letting the model learn what matters, and this combination of learned embeddings and lightweight features helps capture both content and speaking style in a natural way.

Model 3: ELECTRA Features

5 Implementation

Baseline 1: TF-IDF + Logistic Regression

As a baseline, we implemented a Logistic Regression classifier using TF-IDF features. This model achieved an F1 score of 0.23, providing a simple but meaningful benchmark for our project.

Baseline 2: Majority Baseline

Another baseline, we implemented a Majority Vote classifier. This model achieved an F1 score of 0.097.

Model 1: RoBERTa Fine-Tuned Classifier

The first model we implemented was a fine-tuned roberta-base model through Hugging Face. In

addition to the dialogue, we included surrounding context and simple structural features like sentence length and punctuation. This helps the model pick up not just what is being said, but how it is said, which is important for distinguishing between characters.

For training, we used a focal loss function with label smoothing to handle class imbalance and improve learning on harder examples. The model was optimized using AdamW with weight decay, and a linear learning rate scheduler with warmup was applied during training.

Overall, the final model outperformed the TF-IDF + Logistic Regression baseline, demonstrating the benefit of combining contextual transformer embeddings with simple linguistic features.

Model 2: Modern-BERT Fine-Tuned Classifier

The second model we implemented was a fine-tuned ModernBERT-base model from answerdotai through Hugging Face. It was pretrained on 2 trillion tokens of English text, and the context length was up to 8192 tokens. It was also stated that the model is very suitable for processing large documents. (*answerdotai/ModernBERT-base - Hugging Face, 2024*). So, in the case of how diverse each document is for this project, it would be beneficial to the analysis since being trained with a context length this long. Initially, we started out with 3 epochs, a learning rate of $2e-5$, and a train batch size of 8 with no regularization. The dataset was also tokenized with padding and truncation according to the set max length of 128. Then, for further improvement, we updated the training arguments with a regularization, such as setting a weight decay of 0.01. Regularization allowed for a minor improvement in F1 scores but a slight drop in accuracy, however (*by about 0.02*). Furthermore, since we were dealing with classification, the model was defaultly set to use CrossEntropy as a loss function.

Model 3: ELECTRA Fine-Tuned Classifier

The third model that we implemented was a fine-tuned ELECTRA base model (google/electra-base-discriminator) which was pretrained using replaced token detection rather than BERT's approach of masking inputs. This allows the model to produce greater contextual representations, which we were hoping would help produce better results for

our task. When fine-tuning the model, the dataset was tokenized with padding and truncation of max length 128 due to the short nature of television dialogue. Then, the model was trained for 5 epochs with an effective batch size of 8 and learning rate of $2e-5$. After analyzing the outputs and comparing the accuracy and F1 scores with the baseline models, we decided to experiment with implementing class weights due to the low F1 scores. Additionally, we added regularization by setting a weight decay of 0.01 in order to prevent overfitting since we noticed the validation loss gradually increasing over the 5 epochs. Finally, the loss function for the model is cross entropy which is commonly used and suited for the task of multi-label classification.

6 Results and Evaluation

Evaluation

The dataset was split into an 80/10/10 split for training, validation, and testing. Specifically, 80% of the data was used for training, 10% for validation, and 10% for final testing. The validation set was used for model selection and checkpointing, while the test set was used only for final reporting.

We used weighted F1 score as our primary evaluation metric since the dataset is imbalanced across speakers. Accuracy was also reported for comparison with both our's and our classmates's model, but weighted F1 was treated as the main metric because it better reflects performance across all classes.

Comparison to others

The below table summarizes all models evaluated on the test set, including baselines, classmates' approaches, and our implementations.

Model	Accuracy	F1
Baselines		
Majority Class Baseline	0.2459	0.0971 (weighted)
TF-IDF + Logistic Regression	0.2951	0.2304 (weighted)
Classmates' Models		
BoW classifier	0.2696	0.1843 (macro)
DistilBERT Fine-tuned	0.3160	0.1278 (macro)
typeform/distilbert-base-uncased-mnli	0.0087	0.2235 (macro)
google/flan-t5-small	0.0275	0.0187 (macro)
Our Models		
RoBERTa Fine-tuned	0.3232	0.2856 (weighted)
Electra Fine-tuned	0.3010	0.2472 (weighted)
Modern-BERT Fine-tuned	0.3088	0.2645 (weighted)

The baselines showed the expected behavior: the majority class model performed poorly, and TF-IDF with logistic regression gave a modest improvement but was still limited. This confirmed that simple feature-based methods are not sufficient for this task.

Among classmates' models, results were mixed. Some transformer-based approaches struggled significantly, especially zero-shot models, which suggests that label mismatch and lack of task-specific training were major issues. A few fine-tuned models performed better, but they were generally less stable and less consistent across classes.

Our models performed better overall. Fine-tuned transformer models such as RoBERTa, ModernBERT, and ELECTRA consistently outperformed both baselines and most classmates' approaches. RoBERTa gave the strongest and most reliable performance.

A really interesting observation we made was while comparing results for model 3 (ELECTRA) before and after implementing class weights, the model initially performed fairly well in terms of accuracy with a value of 0.3092, which outperforms our simple and trained TF-IDF model baselines and is comparable to the fine-tuned models of our classmates. However, we noticed that the F1 scores were very low especially F1 Macro (F1 Macro: 0.0894, F1 Weighted: 0.2479). This was the case with the baselines and our classmates' models as well and it tells us that our dataset is very imbalanced. Although the F1 scores still outperformed the baselines and most of our

classmates' models, we wanted to experiment with and try to improve our results, so we attempted to retrain the model with the corresponding class weights. After researching how to implement class weights with a custom trainer and retraining the model with the same hyperparameters, the model achieved almost double the F1 Macro score (0.1485) and a slight improvement to the F1 weighted score (0.2513). However, this came at the cost of the models' accuracy which lowered to 0.2365. This makes sense because class weights helps the model predict the classes with less labels better and prevents the model from overly predicting the majority classes. But since the majority classes still make up most of the testing dataset, the accuracy will suffer. Although training the model with class weights helped greatly to improve the model's F1 Macro score, the loss to accuracy is not worth it and our best model is still our fined-tuned RoBERTa model.

These results mostly matched expectations. Transformer models that are properly fine-tuned should outperform traditional baselines, and that is what we observed. The weaker performance of zero-shot models was also expected since the task requires learning dataset-specific patterns rather than general language understanding.

7 Error Analysis

To analyze our model's errors, we examined the confusion matrix shown in Figure 1, along with per-class metrics from the classification report.

The most interesting observation is that the model loves predicting Michael. Honestly, it makes sense since Michael has by far the most lines in the dataset, but it means the model regularly misclassifies characters like Andy, Jim, and Pam as Michael. Characters like Creed, Meredith, and Stanley were the hardest to get right, which is heartbreaking but expected since they simply do not have enough dialogue to learn from.

The model did reasonably well on Michael, Dwight, and Jim since they have the most data and the most distinctive speech patterns. Dwight's blunt, bizarre lines and Michael's over-the-top humour gave the model real signal to work with.

The biggest issue is class imbalance. If we were to continue, we would oversample minority characters or apply stronger class weighting during training. We would also recommend future work explore speaker-specific features like catchphrases or topic tendencies, which could help distinguish the quieter characters that the model currently struggles with.

Despite our efforts to combat class imbalance by experimenting with adding class weights with the ELECTRA model, the experiment ultimately did not produce significant results. Although the experiment was able to increase F1 scores, especially F1 Macro scores, it is not able to breakthrough the inherent issue of class imbalance in the dataset. This issue, along with the difficulty of the task itself causes most models' accuracies to be in the range of around 0.2-0.3 and F1 weighted scores to be around <0.3.

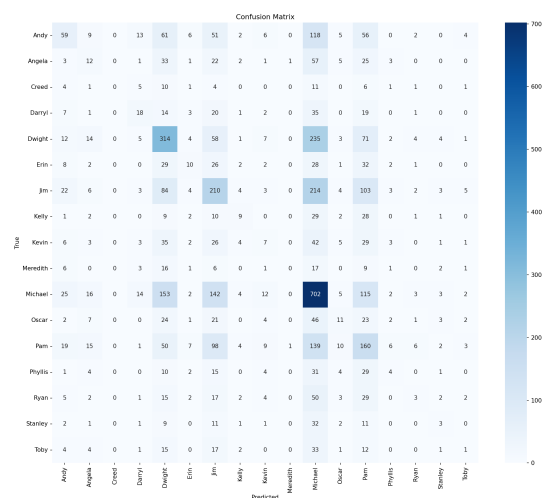


Figure 1: Confusion matrix for our RoBERTa fine-tuned model on the test set.

8 Conclusions

This project started as a simple idea but quickly turned into something much harder than expected. Trying to figure out who said what from short lines of dialogue turned out to be noisy, messy, and often unpredictable. Even with strong models, the dataset itself made the task feel limited at times, especially when some characters barely had enough examples to learn from.

Still, it was clear that the transformer-based models were able to pick up real differences in speaking style, even when those differences were subtle. Our best results came from the RoBERTa-based approach, which consistently felt more stable and reliable than the simpler baselines.

In the end, the biggest takeaway is that the models are not the main bottleneck here. The data is. The imbalance between characters and the lack of context in single lines made the task feel inherently constrained. If anything, this project showed how much real-world noise matters, and how even strong models can struggle when the signal is uneven.

9 Team Contributions

Harsh Upadhyay worked on the fine-tuning and training of the RoBERTa-based classifier. He also wrote the Related Work section and the Error Analysis section.

Kyen So worked on implementing and training the ELECTRA fine-tuned classifier. He also wrote the Results and Evaluation section and the Conclusion section.

Vedant Goyal worked on implementing and training the Modern-BERT fine-tuned classifier. He also wrote the Introduction and Dataset sections, including data collection, preprocessing, and dataset description.

References

- Mandy Fabien, Esau Villatoro-Tello, Petr Motlicek, and Shantipriya Parida. 2022. [Bertaa: Bert fine-tuning for authorship attribution](#). *PMC*.
- Efstathios Stamatatos. 2021. [Authorship attribution using text distortion](#). *Expert Systems with Applications*, 177:114567.
- Jakob Tyo and Sandeep Bhatt. 2024. [Authorship attribution via neural networks](#). *Information*, 15(3):131.
- Max Wiseman. 2024. Office quotes. <https://quotes.maxwiseman.io/episodes>.